

Project Planning

1. Project Title

Monitoring a Containerized URL Shortener Webservice

2. Objective

The main objective of this project is to **build, containerize, and monitor a URL Shortener web service** using Docker, Prometheus, and Grafana.

The service will shorten long URLs, store them in a database, and handle redirects. It will also expose custom performance metrics that can be collected and visualized to monitor the system's health and usage.

Key goals include:

- Building a fully functional URL shortener application.
 - Containerizing the service for easy deployment and scalability.
 - Monitoring performance metrics such as request counts, latency, and errors.
 - Visualizing these metrics through an interactive Grafana dashboard.
-

3. Scope

The project includes the following components:

- Developing a RESTful web service for URL shortening and redirection.
 - Using a lightweight SQLite database to store URL mappings.
 - Containerizing all components using Docker and Docker Compose.
 - Configuring Prometheus to collect metrics from the service.
 - Setting up Grafana to visualize the collected data.
 - Enabling persistent storage for database and monitoring data.
 - Implementing alerting rules to notify when system performance degrades.
-

4. Tools & Technologies

Component	Technology	Purpose
Web Framework	Node.js	Build the REST API for shortening URLs
Database	SQLite	Store mappings between long and short URLs
Containerization	Docker	Package and run the application in containers
Orchestration	Docker Compose	Manage multiple containers (App + Prometheus + Grafana)
Monitoring	Prometheus	Collect performance metrics from the web service
Visualization	Grafana	Display metrics on dashboards
Version Control	Git & GitHub	Manage source code and version control
Documentation	Markdown / Google Docs	Document project details

5. Project Timeline

Week	Tasks	Deliverables
Week 1	Develop the web service (Node.js), set up SQLite DB, create Dockerfile & docker-compose.yml	Functional URL shortener running locally
Week 2	Add Prometheus client metrics (URL count, redirects, latency), configure prometheus.yml	Metrics visible in Prometheus UI
Week 3	Integrate Grafana, connect it to Prometheus, and build a custom dashboard	Complete monitoring dashboard
Week 4	Configure alerts, enable persistent volumes, perform testing, and write documentation	Stable and monitored service with persistence

6. Expected Outcomes

- A functional containerized URL shortener web service.
- Real-time performance monitoring using Prometheus and Grafana.
- Alerting system for detecting anomalies (e.g., high latency or 404 errors).
- Persistent data across restarts using Docker volumes.
- Clear project documentation and API reference.